

Concurrency is basically about dealing with multiple things at once. Not necessarily doing them at the exact same time, but allowing tasks to make progress within the same time period. While parallelism is when tasks are actually running at the same time on different processors.

This chapter explains that java uses threads as the smallest unit of execution, and that thread scheduling is handled by the operating system. This means java doesn't decide exactly when a thread runs it only prioritizes them.

What is a thread?

At first it seemed like the best definition for a thread was the smallest task within a program that can't be broken down any further, but after reading on thread pools threads started to seem like spaces in memory left out for tasks within a program(sort of like a placeholder). So basically I don't completely understand what a thread really is.

The chapter also talks about how threads are executed. Instead of manually creating and managing threads, it encourages us to use the Executor framework. By implementing runnable and submitting tasks to an executor service, you separate what work needs to be done from how it's executed. The executor service manages thread pools for you, which makes concurrency safer and easier to reason about.

Synchronization is introduced as a way to protect shared data. Java objects have monitors and monitor locks, which ensure that only one thread can access critical sections at a time. This is important because most concurrency problems happen when multiple threads modify the same data without coordination.

Basically the chapter pushes that concurrency is hard mostly because of shared data and unpredictable timing and java gives us tools to manage this, but the safest approach is to rely on high-level concurrency utilities instead of trying to control everything manually.